

To implement this hybrid security model, the best architectural approach is to stop copy-pasting the checksums class into every single file (as seen in the original nmapWrapper.py, bettercap.py, blueTooth.py, etc.).

Instead, we should create a single, centralized security.py module inside the core/ directory. This module will house both the dpkg (system package) verification and the hashlib (custom script) verification. Then, the individual tools simply import and call the appropriate method. Here is the refactored code to achieve this.

1. Create a Centralized Security Module (source/core/security.py)

This file handles both system package verification and native Python hashing.

```
import hashlib
import subprocess
import os
import json

install_location = f'{os.getenv("HOME")}/.SuperSploit'
CHECKSUMS_FILE = f'{install_location}/.data/.security/checksums.json'

class SecurityValidator:
    def __init__(self):
        self.checksums = self._load_checksums()

    def _load_checksums(self):
        if not os.path.exists(CHECKSUMS_FILE):
            return {}
        try:
            with open(CHECKSUMS_FILE, "r") as file:
                return json.load(file)
        except Exception:
            return {}

    @staticmethod
    def verify_system_package(package_name: str) -> bool:
        """
        Verifies integrity of system packages installed via apt/dpkg.
        Returns True if the package is unmodified, False if tampered
        or missing.
        """
        try:
            # dpkg -V returns 0 if all files in the package pass
            integrity checks
            result = subprocess.run(
                ["dpkg", "-V", package_name],
                capture_output=True,
                text=True
            )
            return result.returncode == 0
```

```

        except FileNotFoundError:
            # Fallback if dpkg is not available on the OS
            return False

    @staticmethod
    def get_sha256(file_path: str) -> str:
        """Securely generates a SHA256 hash using native Python
hashlib."""
        if not os.path.isfile(file_path):
            return ""

        sha256_hash = hashlib.sha256()
        try:
            # Read in chunks to prevent memory overload on large
binaries
            with open(file_path, "rb") as f:
                for byte_block in iter(lambda: f.read(4096), b""):
                    sha256_hash.update(byte_block)
            return sha256_hash.hexdigest()
        except Exception:
            return ""

    def verify_custom_binary(self, tool_name: str, file_path: str) ->
bool:
        """Verifies a custom downloaded binary against the recorded
hash."""
        expected_hash = self.checksums.get(tool_name, "")
        if not expected_hash:
            return False

        actual_hash = self.get_sha256(file_path)
        return actual_hash == expected_hash

# Global instance for easy importing
validator = SecurityValidator()

```

2. Updating System Tools (e.g., nmapWrapper.py)

For tools managed by the OS like Nmap, Bettercap, and Recon-ng, we strip out the old `.encode()` hash checking and rely entirely on `dpkg`. This makes your code much cleaner and immune to false positives from OS updates.

Changes in `source/core/reconCore/networkRecon/nmapWrapper.py`:

```

# Import the new centralized validator
from ...security import validator

# ... [imports and setup] ...

```

```

class nmap:
    # ... [init and other methods] ...

    def scan_whole_network(self) -> str:
        # NEW: Use the package manager to verify nmap instead of a
        hardcoded hash
        print("[*] Verifying Nmap integrity via package manager...")
        if not validator.verify_system_package("nmap"):
            print("[!] Integrity verification Failed! The system
package 'nmap' may be tampered with.")
            if not input_prompt("[*] Would you still like to proceed
[y/n]: ").lower().endswith("y"):
                return "[*] Scan aborted."
            else:
                print("[*] Nmap binary integrity verified successfully.")

        # ... [rest of the nmap scanning logic] ...

```

3. Updating Downloaded/Custom Scripts (e.g., phoneinfoga.py or BlueDucky.py)

For tools that are downloaded directly from GitHub (like Phoneinfoga) or your own internal scripts (like BlueDucky.py), we use the native hashlib method.

Changes in source/core/reconCore/external_tools/phoneinfoga.py:

```

import os
from subprocess import run
from prompt_toolkit import PromptSession
from prompt_toolkit.history import FileHistory
from prompt_toolkit.auto_suggest import AutoSuggestFromHistory

# Import the centralized validator
from ...security import validator

installation = f'{os.getenv("HOME")}/.SuperSploit'
# ... [input session setup] ...

class Phone:
    def __init__(self, char):
        try:
            tool_path = "/usr/local/bin/phoneinfoga"
            print("[*] Verifying Phoneinfoga binary integrity...")

            # NEW: Use native Python hashing for custom binaries
            if not validator.verify_custom_binary("phoneinfoga",
tool_path):
                print(f"[!] Checksum verification Failed!")

```

```

        print(f"[*] Expected:
{validator.checksums.get('phoneinfoga')}")
        print(f"[*] Actual:
{validator.get_sha256(tool_path)}")
        if not input("[*] Would you still like to proceed
[y/n]: ").endswith("y"):
            return
        else:
            print("[*] Checksum verified successfully.")

# ... [rest of phoneinfoga logic] ...

```

Why this Architecture is Better:

1. **DRY Principle (Don't Repeat Yourself):** You eliminate the ~15 lines of checksums class boilerplate that was duplicated across 5 different files.
2. **Maintenance Free System Tools:** You never have to manually update the nmap or bettercap hashes in checksums.json again. As long as apt upgrade updates the tool, dpkg knows it's legitimate.
3. **No Shell Subprocessing for Hashes:** By using hashlib, you eliminate the risk of an attacker placing a fake sha256sum script in the system \$PATH to bypass your security checks.